Optimizing Sparse/Dense VEGETA Accelerator Performance with Microscaling Quantization

Kazi Barria Nine[1], Connor Talley[1], Ajay Sharma Mandadi[1], Tushar Krishna[1], and Arijit Raychowdhury[1]
[1]Georgia Institute of Technology
[1]knine3@gatech.edu, connor.talley@gatech.edu, amandadi3@gatech.edu,
tushar@ece.gatech.edu, arijit.raychowdhury@ece.gatech.edu

2025 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3503-5683-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCAS56072.2025.11043957

***Abstract*—Recent advancements in artificial intelligence have led to the development of exponentially larger models, which in turn require substantial amounts of memory. This escalating demand presents a challenge to scaling large models efficiently. One potential solution is quantization, which leverages low bit-width precision to mitigate memory issues. In this study, we explore the integration of low bit-width Microscaling (MX) based quantization into the VEGETA systolic array design. To achieve this, we introduce dedicated buffers within the systolic array architecture to multicast scale factors for both inputs and weights across tiles. Additionally, we modify the adders and multipliers to function using Microscaled floating point (MXFP) formats. SCALE-Sim is adapted to verify our approach, and results indicate that the MXFP format effectively addresses the memory-wall problem, providing a 1.3× speed-up for weight stationary dataflow. In terms of hardware, the MXFP6 (E3M2) implementation delivers significant benefits, including a 4.65× reduction in area, 2.45× lower power consumption, and a 53% increase in maximum operating frequency. Moreover, it achieves an impressive energy efficiency of 7.912 TOPS/W. In conclusion, Microscaling offers a viable and efficient solution to overcome the memory scaling challenges inherent to large AI models.**

***Index Terms*—Quantization, Microscaling, Systolic array, memory-wall**

I. Introduction

With the recent surge in applications in the field of Artificial Intelligence (AI), the demand for higher memory capacity and increased computational power has grown exponentially. This rapid advancement in AI technologies necessitates more robust hardware solutions to meet the ever-increasing performance requirements across various AI-driven domains [1].

To address this challenge, some of the compression methods such as pruning, knowledge distillation and quantization are becoming popular [2]. Though pruning redundant parameters can enhance model's storage efficiency, memory utilization, and computational efficiency [3], it can heavily affect the accuracy and require large training pipelines [4]. On the other hand, Knowledge Distillation (KD) refers to transferring knowledge from a sophisticated model, often called the 'teacher model', to a more straightforward counterpart known as the 'student model' [5]. That being said, finding a proper stable student model is quite a challenge [6].

In this regard, quantization presents a potential solution. It involves the conversion of data from full precision (FP32) formats to lower bit representations such as BF16, FP8, FP6, thereby reducing the computational complexity and memory requirements. In recent literature, both Quantization-Aware Training (QAT) [7] and Post-training Quantization (PTQ) [8] have gained significant popularity as effective techniques. Though QAT can have a complex training pipeline, PTQ can be a viable option and can work efficiently on an accelerator.

However, heavily quantized data formats (e.g. 8-bit) generally require dynamic tensor-level scaling factors in order to adjust the range of numbers representable by elements within the tensor to minimize information loss [9]. While this generally works well at 8-bit or higher, the even more limited range of sub-8 bit data formats can lead to significant accuracy loss under tensor-level scale factors [10]. To address this issue, recent approaches, specifically Microscaling (MX) data formats [11], have demonstrated that employing scale factors at a finer granularity is more effective in mitigating accuracy loss.

To further optimize computational efficiency and performance, state-of-the-art research indicates that sparsity-supported accelerators offer significant benefits. By bypassing ineffectual computations, such as multiplication by zero, these accelerators reduce memory capacity and bandwidth needs by storing only non-zero values [12]. A very recent work, VEGETA [13], provides substantial advantages for deep learning acceleration on CPUs by incorporating flexible N:M structured sparsity support on matrix engine through ISA and microarchitecture extensions. This leads to improved performance and reduced memory requirements. Comparative analyses show significant performance gains when executing sparse DNN layers, highlighting VEGETA's effectiveness in enhancing overall computational efficiency.

In this paper, we have incorporated Microscaling quantization support into a VEGETA-based accelerator. By adding effective MX quantization with VEGETA sparse acceleration, we can further push the boundary of memory reduction while also accelerating computation. The key contributions of this paper are as follows:

- The integration of low bit-width Microscaling quantization support for sparsity-aware VEGETA systolic array hardware design.
- The implementation of shared per tile scaling factors, reducing the need for a larger scaling factor buffer.
- The extension of SCALE-Sim [14] to model Microscaling quantization and understand the memory-bandwidth and compute requirements.
- The evaluation of the accelerator's resulting increase in hardware compute efficiency and lower area cost.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:05:45 UTC from IEEE Xplore. Restrictions apply.

II. Background and Motivation

A. Microscaling

In the MX format, the fundamental component is the MX block, comprising a vector of k numbers. Each MX block consists of a singular shared scale denoted by X, along with k scalar elements $P_{i=1}^{k}$. The structure of an MX block is defined by the combination of its block size k, scale data format, and element data format. Notably, the scale data format and element data format are independent of each other, with all k elements adhering to the same element data format. The Microscaled (MX) values are represented as follows [11]:

- Let, $v_{i}{}_{i=1}^{k}$ represent the k real numbers encoded within an MX block.
- If $X = \text{NaN}$, then $v_i = \text{NaN}$ for all i.
- If $|XP_i| > V_{\max_{\text{Float32}}}$, then v_i is implementation-defined.
- Otherwise, $v_i = XP_i$.

B. VEGETA Architecture

1) Processing Unit (PU): A VEGETA Processing Unit (PU) consists of multiple MAC units that contribute to the same output element. In a weight-stationary (WS) systolic array, the dot product for each output is mapped to a column of PUs. Partial sums propagate down the column, with the final result emerging at the bottom. The dot product operations in VEGETA sparse accelerator is further divided inside each PU, allowing multiple MAC units to be packed into each PU and thus for increased parallel processing. At the bottom, partial sums from each lane is combined. For MAC units, the original VEGETA implementation uses Bf16 multiplication and FP32 addition. The number of lanes or MAC units per PU, known as the reduction factor β determines how many partial sums are reduced to produce a single output element.

2) Processing Element (PE): VEGETA Processing Units (PUs) sharing the same eastbound inputs and output buffers are grouped into a Processing Element (PE). Inputs from the west are broadcast to all PUs in a PE. The number of PUs per PE is the broadcast factor α, and configurations are labeled as PE-α-β. Depending on the format and supported degree of sparsity, we can choose the appropriate broadcast factor.

III. Implementation

In our proposed architecture, we combine traditional VEGETA sparse PEs ($\alpha = 2$, $\beta = 2$) with modified Microscaling compatible FP MAC unit and adder tree. We also extend our work for SCALE-Sim to verify our initial assumption of increased computation efficacy of lower precision numbers, as well as supplement our compute implementation with memory simulation results. Details of our implementation are described in this section.

A. Microscaling Format

In our method, we use traditional VEGETA sparse PEs ($\alpha = 2$, $\beta = 2$) with modified FP MAC unit. Details of our implementation are described in the following sections.

1) Precision: It has been previously found that MX formats deliver significant efficiency improvements over FP32, while maintaining similar accuracy [11]. Keeping the accuracy trade-off in mind MXFP6, MXFP8 is found to be a suitable replacement for traditional FP32 format after software fine tuning. MXFP4 has significant accuracy losses when compared to FP32. For this reason, we pursue our quantization approach using MXFP8 and MXFP6. Details of the implemented precisions are listed in table I.

TABLE I
Implemented MX Formats

MX Format	Data Format	Bit-Width	Scale Bits
MXFP6	FP6 (E2M3)	6	8
MXFP8	FP8 (E4M3)	8	8
MXFP8	FP8 (E5M2)	8	8

B. SCALE-Sim Extension

SCALE-Sim [14] is an open-source, cycle-accurate DNN accelerator simulator. It offers a comprehensive model for a functionally accurate systolic array with a well defined memory hierarchy of SRAM/DRAM buffers for input activations, filter weights, and output activations. It also offers flexibility in running different data flow patterns such as output stationary, input stationary, and weight stationary for different matrix sizes and supports different matrix operations (convolutions, GEMM, etc). However, SCALE-Sim's baseline implementation does not consider the data format of the inputs and assumes each data to be a single word unit.

Here, we design an extended version of SCALE-Sim to model a representation of the MX format that accepts it as an input and integrates different word sizes into the simulator to give a more accurate analysis of compute and bandwidth. This gives a detailed understanding of how varying data format effects SRAM and DRAM bandwidths for the different sized buffers along with different dataflow techniques. The modifications also elaborate on how stall cycles due to memory access patterns changes and results in different total compute cycles.

It is important to note here that the software evaluation does not simulate the various data formats for MAC units in the systolic arrays and assumes that for each data format the necessary optimized MAC is present. The goal of software evaluation is to understand the memory-bandwidth impacts of different data formats on different buffer sizes with various matrix operators and how this effects the overall running time of the operations.

C. Modified Architecture

As SCALE-Sim's extension for our work verifies our assumed memory-wall solution for low-precision quantization, we dive into hardware assessment. The Microscaling-aware systolic array implementation shares most of its behavior with a traditional systolic array. We build off of the VEGETA [13] implementation of a systolic array for our implementation,

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:05:45 UTC from IEEE Xplore. Restrictions apply.

with plans to leverage the added sparsity of quantization with VEGETA's sparsity support to further increase performance benefits in the future. As such, the systolic array employs a weight-stationary dataflow, in which the weights/filter elements are streamed into and stored in the PEs before input feature map rows are streamed across the PEs to obtain the output partial sums of the matrix multiplication. An example of our modified architecture with a 3×3 per-tile scaling factor granularity can be seen in Fig. 1.

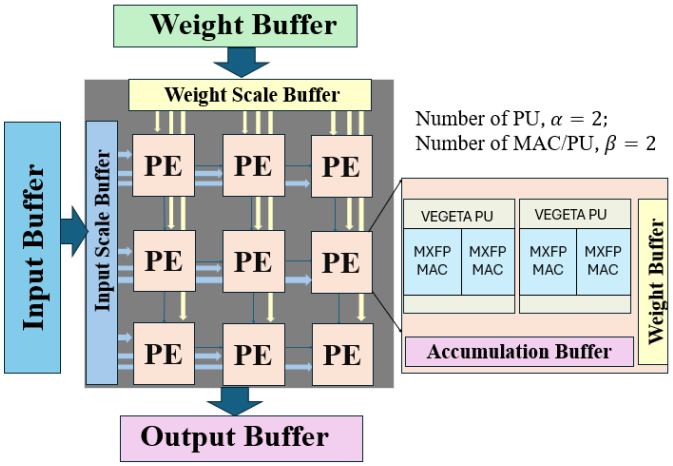


Fig. 1. Example Microscaled VEGETA Implementation for 3×3 Array with shared weight and input scale factor buffers for per tile MX implementation.

To support the functionality of operating on the scale factors in parallel with the relevant data needed for the output feature map, we create two scale factor buffers for the inputs and weights of the current tile that multicast the value of the scale factors to the PEs for use in computations. The design can be extended to support finer granularity of Microscaling, at the row and sub-row level.

Implementation is done TSMC's 65LP technology node for a 16×8 array of PEs. Each PE comprises of 2 PUs, and each PU has 2 MAC units ($\alpha = 2$, $\beta = 2$), for a total of 512 MAC units. This can be converted to dense format by choosing $\alpha = 1$ and $\beta = 1$. We modify our MAC unit to support MXFP multiplication with scale factor incorporation. We incorporate weight stationary dataflow in this case. We use FP6/FP8 MAC units to multiply quantized FP6/FP8 inputs and weights. Fig. 2 shows modified MAC unit. In this system, both input and output scale factors are represented by exponents (E8M0 format). As a result, multiplication in the exponent domain is effectively an addition operation. Consequently, this introduces an additional ADD operation in the computational process.

To maintain consistency with the original Microscaling approach, the adder tree is kept in BF16 format. The result of the multiplication is scaled up and then multiplied by the output scale factor before being added to the input accumulation.

IV. Evaluation and Results

A. SCALE-Sim

The extended version of SCALE-Sim is used to design a simple systolic array and issue a single Matrix Multiply

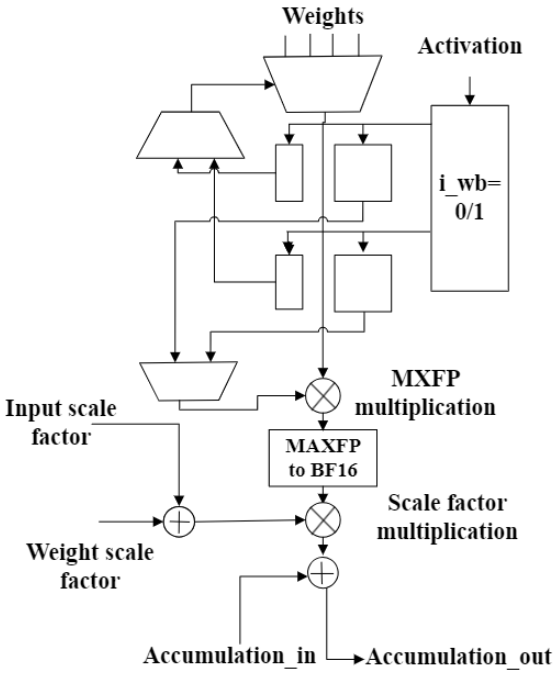


Fig. 2. Modified MAC unit supporting Microscaling format operations. The MXFP data and scale factors for the inputs and weights are operated on in parallel for the multiplication before the result is converted to FP for the accumulation.

operation. We use a 3x3 systolic array as the base compute with a total of 9 PEs and considered the IFMAP, FILTER and OFMAP SRAM buffers to be of size 1kB each. The off-chip DRAM bandwidth is set to be 1MB/s with a single memory bank. For the matrix multiply operation, an input matrix of size [M, N] = [32, 128] and a filter matrix of size [N, K] = [128, 64] are considered with the intention being to capture the various effects of data-formats as well as dataflow techniques.

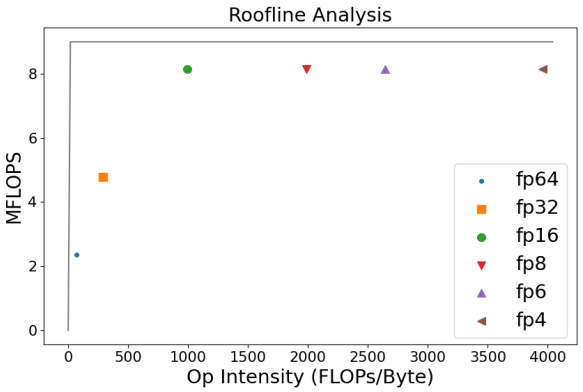


Fig. 3. Roofline plot for matmul operation showing the effect of data-format on being memory vs compute bound 1kB buffers and 1MB/s bandwidth.

Fig. 3 shows the impact of having different data-formats on memory vs compute boundedness of the matmul operation. We note that fp64 is heavily memory bound, but as the data is scaled down there is a shift in the operation moving away from being memory bottle-necked to being more compute intensive. This is because the same bandwidth supports more data elements to be transferred from memory and gives the

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:05:45 UTC from IEEE Xplore. Restrictions apply.

PE's a greater number of elements to process without having to wait for long latency DRAM accesses.

Furthermore, as we decrease data size, the total cycles needed also decreases. This is mainly due to the reduction in the number of stall cycles caused due to high latency DRAM responding to memory requests. We notice that using fp16 or lower gives a speedup of 3.4x for output-stationary dataflow and 1.3x for weight-stationary dataflow. There is a saturation from fp16 and lower as a result of being compute bound for the example input. The saturation point varies based on operator intensity, with lower-reuse operators requiring smaller data formats to achieve saturation.

B. Microscaled VEGETA

From evaluation result observed from SCALE-Sim extension, it is visible that lower precision floating point provides us with good solution of memory bound issues. We implement lower precision Microscaled VEGETA architecture on TSMC's 65LP node to quantify further area, power, and performance improvement.

TABLE II
MAX FREQUENCY AND AREA COMPARISON

Precision	**Maximum Frequency(MHz)**	**Area(mm^2)**	**NAND2 Eq. Area**
Baseline	117.65	3.96	2,752K
MXFP8(E5M2)	180.18	1.01	701K
MXFP8(E4M3)	180.18	1.32	909K
MXFP6(E3M2)	180.18	0.851	591K

Table II shows area, NAND2 equivalent area and maximum frequency achieved compared to baseline VEGETA. Maximum frequency achived for each of the MXFP precision is 180.18 MHz, which is 53% increase from baseline. We note that the maximum frequency did not further increase depending on the MXFP format due to multicasting the shared scale factor becoming the bottleneck, as opposed to the computation itself. For area, the highest reduction is achieved for MXFP6 which is 4.65×, and the lowest area reduction is achieved for MXFP8 (E4M3) being 3× less than the baseline. The area differences observed between the two MXFP8 formats are due to the size of the FP MAC unit being mostly dependent on the number of mantissa (M) bits. We also get the NAND2 equivalent area measurement from our implementation which can provide an idea of the area independent of the technology node. From table II, it is visible that MXFP6 format has lowest NAND2 equivalent area.

Power and energy efficiency comparison of Microscaled VEGETA implementation with baseline is tabulated in table III. It can be noticed that all of the MX implementations provide significant improvements, with faster clock speeds and better energy efficiency per operation. Highest efficiency 7.912 $TOPS/W$ is achieved for MXFP6 format, which is a 3.74× improvement from the baseline. Both configuration of MXFP8 precision also provides with remarkably higher efficiency with significantly decreased power consumption at a higher operating frequency.

TABLE III
POWER AND ENERGY EFFICIENCY COMPARISON

Precision	**Power (mW)**	**Energy Efficiency (TOPS/W)**
Baseline	57.0073	2.11
MXFP8(E5M2)	28.57	6.46
MXFP8(E4M3)	32.44	5.68
MXFP6(E3M2)	23.194	7.912

V. LIMITATIONS AND FUTURE WORK

As noted in Table II, multi-casting the shared scale factor limited the achievable frequency. This limits the ability to scale the design to larger array sizes. We project that this limitation can be mitigated by employing a finer granularity in the shared scale factor, such as supporting multiple scale factors per tile, ensuring more efficient resource utilization and data distribution. We also rely on Scale-Sim to model the benefits of the quantization outside of the compute. Implementing the memory alongside the compute would give more accurate estimates on memory benefits, as well as open possibilities for more optimization.

VI. CONCLUSION

Quantization of data can offer multiple key advantages. The extended SCALE-Sim software model shows that with low-precision quantization, memory intensive operations can be sped up to have more effective bandwidth usage and can represent the input data using much smaller sized buffers. This also improves performance by reducing the number of stall cycles waiting on memory. This result verifies our assumption further of Microscaled quantization being a probable memory-wall solution. It is visible from SCALE-Sim evaluation that lower precision can give $1.3\times$ to $3.4\times$ speed up depending on dataflow format.

Most of the quantization method advantages have previously been primarily realized in memory, whereas, we develop a quantization supported systolic array that takes advantage of quantization for compute performance. With multicasting support it also reduces the need of larger buffer area. All of the MXFP formats that we evaluate provide significant improvements over baseline. Highest improvement is achieved for MXFP6 format where we are able to demonstrate a 53% increase in maximum frequency at $2.45\times$ less power draw and a $4.65\times$ reduction of area. Highest energy efficiency is achieved to be 7.912 $TOPS/W$.

VII. ACKNOWLEDGMENT

This work was funded by CoCoSys, one of the seven center of Joint University Microelectronics Program (JUMP 2.0). We thank Professor Vishvesh Sathe and Zheping Yin from ECE, Georgia Tech for providing us with corrected version of Baseline VEGETA accelerator. Finally, we thank Geonhwa Jeong et. al for creating VEGETA sparse accelerator.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:05:45 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. S. Gill, H. Wu, P. Patros, C. Ottaviani, P. Arora, V. C. Pujol, D. Haunschild, A. K. Parlikad, O. Cetinkaya, H. Lutfiyya, *et al.*, "Modern computing: Vision and challenges," *Telematics and Informatics Reports*, p. 100116, 2024.

[2] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Journal of Machine Learning Research*, vol. 24, no. 5, pp. 1234–1256, 2023.

[3] A. Ardakani, Z. Ji, S. C. Smithson, B. H. Meyer, and W. J. Gross, "Learning recurrent binary/ternary weights," in *7th International Conference on Learning Representations, ICLR 2019*, (New Orleans, LA, USA), OpenReview.net, May 6-9 2019.

[4] B. Li, "Compression of deep neural network," in *Embedded Artificial Intelligence: Principles, Platforms and Practices*, pp. 75–94, Springer, 2024.

[5] F. Tung and G. Mori, "Similarity-preserving knowledge distillation," in *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019*, (Seoul, Korea (South)), pp. 1365–1374, IEEE, October 27 - November 2 2019.

[6] Y.-W. Hong, J.-S. Leu, M. Faisal, and S. W. Prakosa, "Analysis of model compression using knowledge distillation," *IEEE Access*, vol. 10, pp. 85095–85105, 2022.

[7] U. Kulkarni, S. Meena, P. Joshua, K. Kruthika, G. Platel, K. Jadhav, and A. Singh, "Performance improvements in quantization aware training and appreciation of low precision computation in deep learning," in *Advances in Signal Processing and Intelligent Recognition Systems: 6th International Symposium, SIRS 2020, Chennai, India, October 14–17, 2020, Revised Selected Papers 6*, pp. 90–107, Springer, 2021.

[8] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27168–27183, 2022.

[9] B. Darvish Rouhani, D. Lo, R. Zhao, M. Liu, J. Fowers, K. Ovtcharov, A. Vinogradsky, S. Massengill, L. Yang, R. Bittner, *et al.*, "Pushing the limits of narrow precision inferencing at cloud scale with microsoft floating point," *Advances in neural information processing systems*, vol. 33, pp. 10271–10281, 2020.

[10] S. Dai, R. Venkatesan, M. Ren, B. Zimmer, W. Dally, and B. Khailany, "Vs-quant: Per-vector scaled quantization for accurate low-precision neural network inference," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 873–884, 2021.

[11] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V. Elango, M. Golub, A. Heinecke, P. James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P. Dubey, P. Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung, "Microscaling data formats for deep learning," 2023.

[12] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "Extensor: An accelerator for sparse tensor algebra," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 319–333, 2019.

[13] G. Jeong, S. Damani, A. R. Bambhaniya, E. Qin, C. J. Hughes, S. Subramoney, H. Kim, and T. Krishna, "Vegeta: Vertically-integrated extensions for sparse/dense gemm tile acceleration on cpus," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 259–272, 2023.

[14] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "Scale-sim: Systolic cnn accelerator simulator," 2019.

Authorized licensed use limited to: Georgia Institute of Technology. Downloaded on November 06,2026 at 18:05:45 UTC from IEEE Xplore. Restrictions apply.